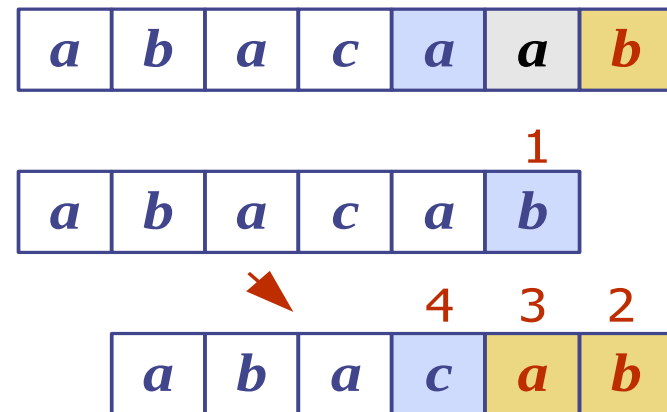


Lecture 11

Text Processing



Learning Outcome

◆ Pattern matching algorithms

- Brute-force algorithm
- Boyer-Moore algorithm
- Knuth-Morris-Pratt algorithm

◆ Trie

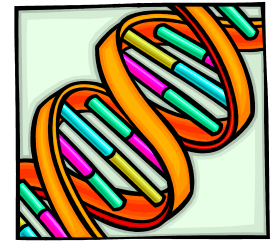
- Huffman encoding

Strings



- ◆ A string is a sequence of characters
- ◆ Examples of strings:
 - Java program
 - HTML document
 - DNA sequence
 - Digitized image
- ◆ An alphabet Σ is the set of possible characters for a family of strings
- ◆ Example of alphabets:
 - ASCII
 - Unicode
 - $\{0, 1\}$
 - $\{A, C, G, T\}$

- ◆ Let P be a string of size m
 - A substring $P[i..j]$ of P is the subsequence of P consisting of the characters with ranks between i and j
 - A prefix of P is a substring of the type $P[0..i]$
 - A suffix of P is a substring of the type $P[i..m-1]$
- ◆ Given strings T (text) and P (pattern), the pattern matching problem consists of finding a substring of T equal to P
- ◆ Applications:
 - Text editors
 - Search engines
 - Biological research



Brute-Force Algorithm

- ◆ The brute-force pattern matching algorithm compares the pattern P with the text T for each possible shift of P relative to T , until either
 - a match is found, or
 - all placements of the pattern have been tried
- ◆ Brute-force pattern matching runs in time $O(nm)$
- ◆ Example of worst case:
 - $T = aaa \dots ah$
 - $P = aaah$
 - may occur in images and DNA sequences
 - unlikely in English text

Algorithm *BruteForceMatch*(T, P)

Input text T of size n and pattern P of size m

Output starting index of a substring of T equal to P or -1 if no such substring exists

for $i \leftarrow 0$ **to** $n - m$

{ test shift i of the pattern }

$j \leftarrow 0$

while $j < m \wedge T[i + j] = P[j]$

$j \leftarrow j + 1$

if $j = m$

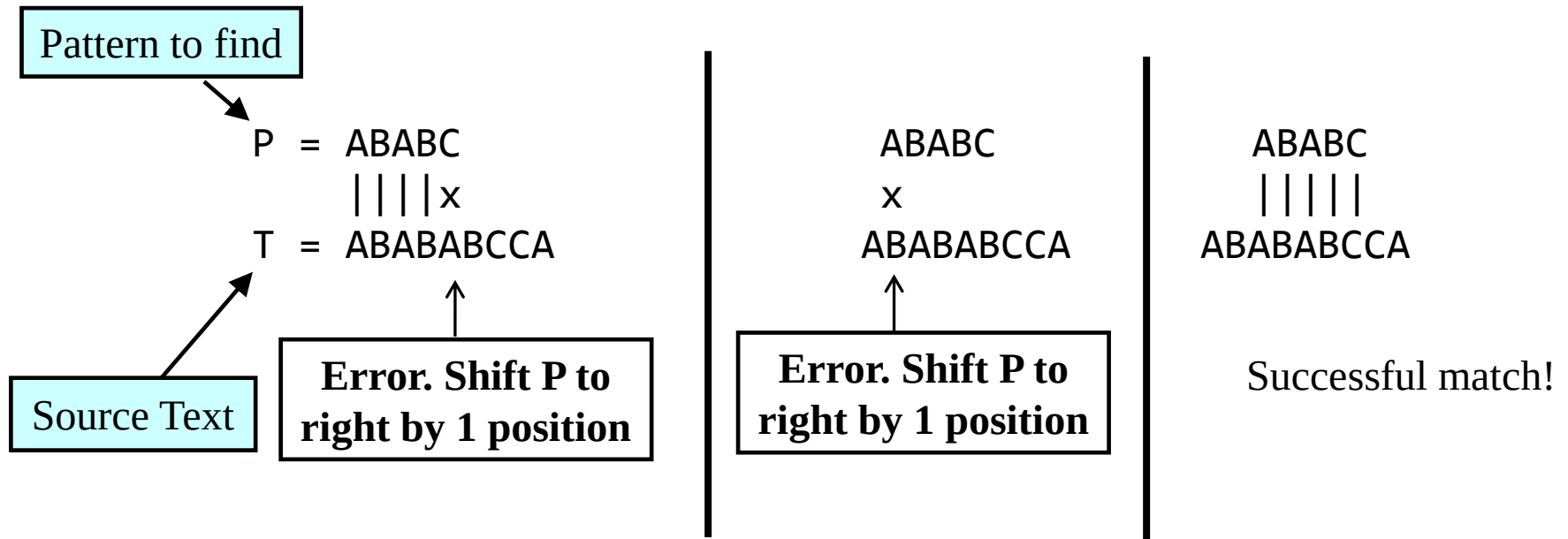
return i {match at i }

return -1 {no match anywhere}

Brute-Force Algorithm

Matching:

Brute-Force Matching:
Comparing from left to right



Brute Force Algorithm

- ◆ Brute-Force Matching II

- ◆ Worst case looks like:

Text: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaab
Pattern: aaaab

- ◆ Best case looks like:

Text: aaaabjhgasjhgasjhagsjhagsjhagahgaaaaaaaaab
Pattern: aaaab

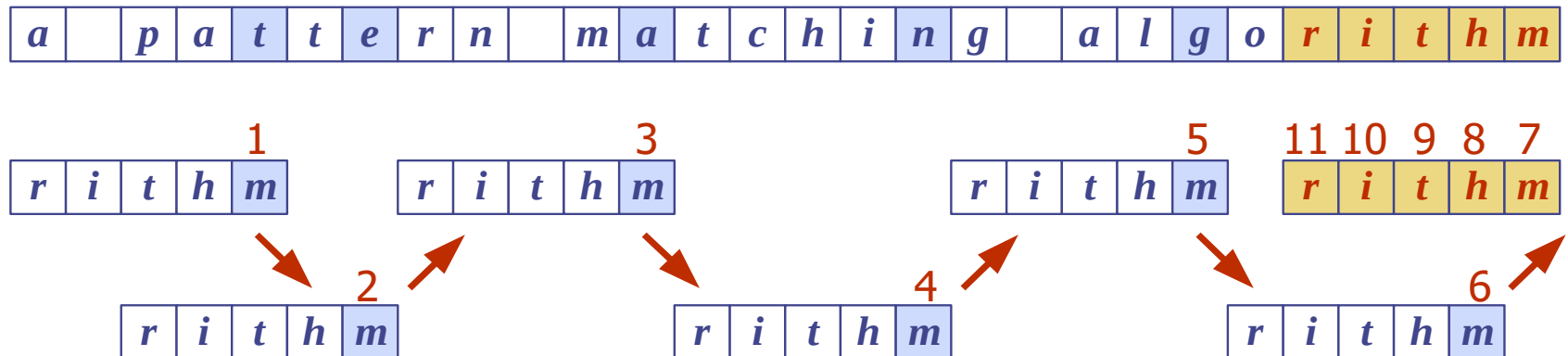
- ◆ Typical average case:

Text: This is an algorithm difficult to beat
Pattern: algorithm

- ◆ Analysis: Worst Case: $O(mn)$
 Best Case: $O(m)$
 Average Case: $O(m + n)$

Boyer-Moore Heuristics

- ◆ The Boyer-Moore's pattern matching algorithm is based on 2 heuristics:
 1. Looking-glass heuristic: Compare P with a subsequence of T moving backwards
 2. Character-jump heuristic: When a mismatch occurs at $T[i] = c$
 - If P contains c , shift P to align the last occurrence of c in P with $T[i]$
 - Else, shift P to align $P[0]$ with $T[i + 1]$
- ◆ Example



Last-Occurrence Function

- ◆ Boyer-Moore's algorithm preprocesses the pattern P and the alphabet Σ to build the last-occurrence function L mapping Σ to integers, where $L(c)$ is defined as
 - the largest index i such that $P[i] = c$ or
 - -1 if no such index exists

- ◆ Example:

- $\Sigma = \{a, b, c, d\}$
- $P = abacab$

c	a	b	c	d
$L(c)$	4	5	3	-1

- ◆ The last-occurrence function can be represented by an array indexed by the numeric codes of the characters
- ◆ The last-occurrence function can be computed in time $O(m + s)$, where m is the size of P and s is the size of Σ

The Boyer-Moore Algorithm

Algorithm *BoyerMooreMatch*(T, P, Σ)

$L \leftarrow \text{lastOccurrenceFunction}(P, \Sigma)$

$i \leftarrow m - 1$

$j \leftarrow m - 1$

repeat

 if $T[i] = P[j]$

 if $j = 0$

 return i { match at i }

 else

$i \leftarrow i - 1$

$j \leftarrow j - 1$

 else

 { character-jump }

$l \leftarrow L[T[i]]$

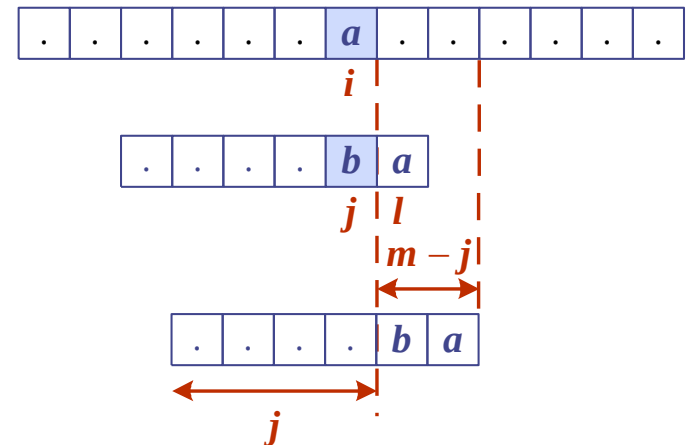
$i \leftarrow i + m - \min(j, 1 + l)$

$j \leftarrow m - 1$

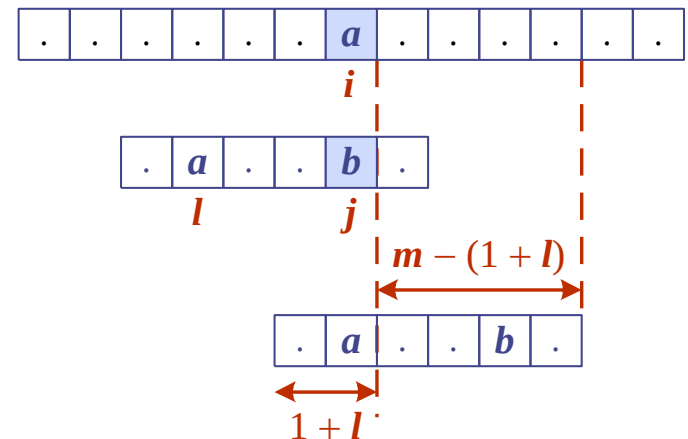
until $i > n - 1$

return -1 { no match }

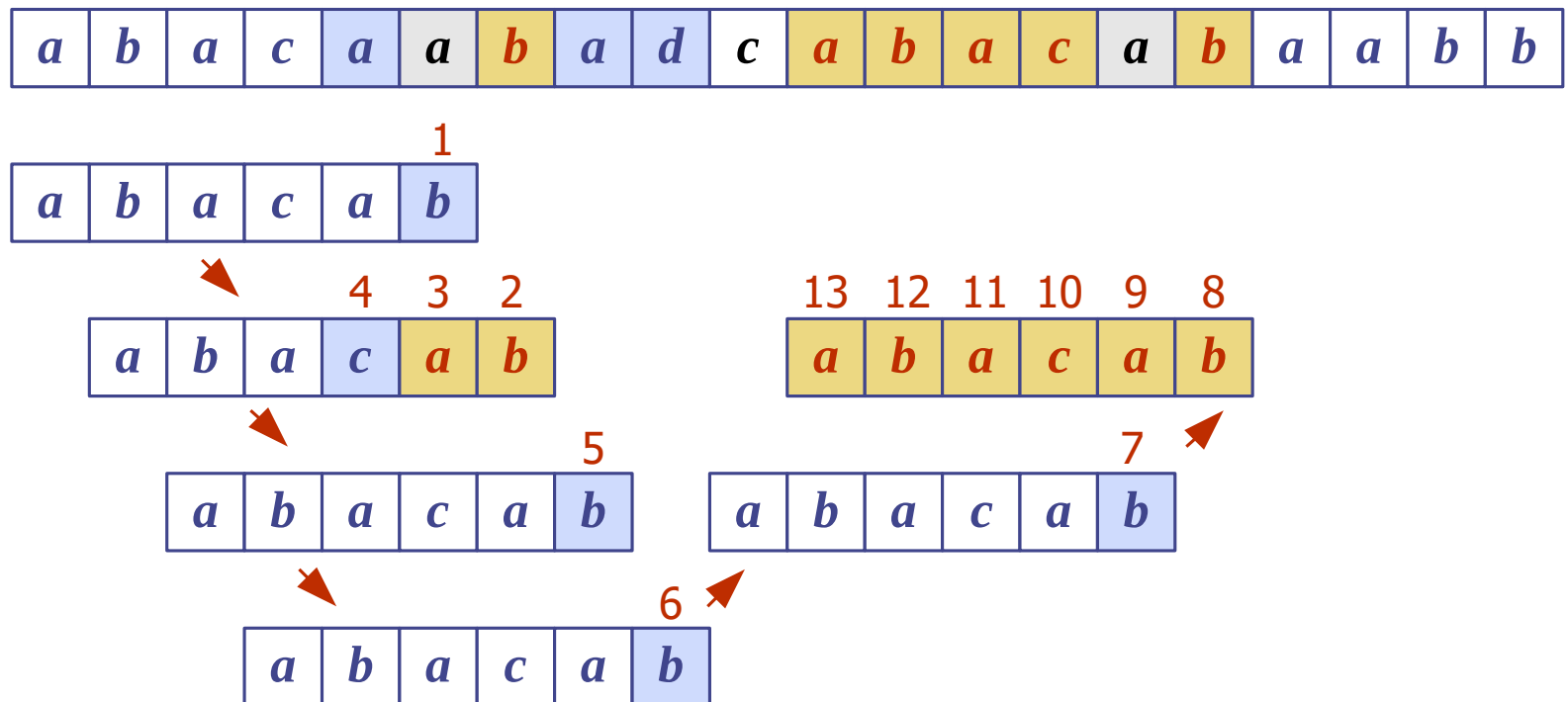
Case 1: $j \leq 1 + l$



Case 2: $1 + l \leq j$



Boyer-Moore Example (right to left)



Another Boyer-Moore Example

must

x

If you wish to understand others you must intensify ..

No match and 'y' is not in must, so move the whole word.

must

x

If you wish to understand others you must intensify ..

No match and 'w' is not in must, so move the whole word.

must

x

If you wish to understand others you must intensify ..

No match and ' ' is not in must, so move the whole word.

Another Boyer-Moore Example

must

x

If you wish to understand others you must intensify ..

No match, but 'u' is in must, so move the whole word only to right until u fits.

must

x

If you wish to understand others you must intensify ..

No match and 'd' is not in must, so move the whole word.

must

x||

If you wish to understand others you must intensify ..

No match and 'r' is not in must, so move the whole word.

must

x

If you wish to understand others you must intensify ..

No match and 'n' is not in must, so move the whole word.

Another Boyer-Moore Example

The rest goes this way:

```

                                must
                                |||
                                must
                                x
      must      x      x
    must  x    x      x
  must  x  x  x      x
x|  x  x  x      x

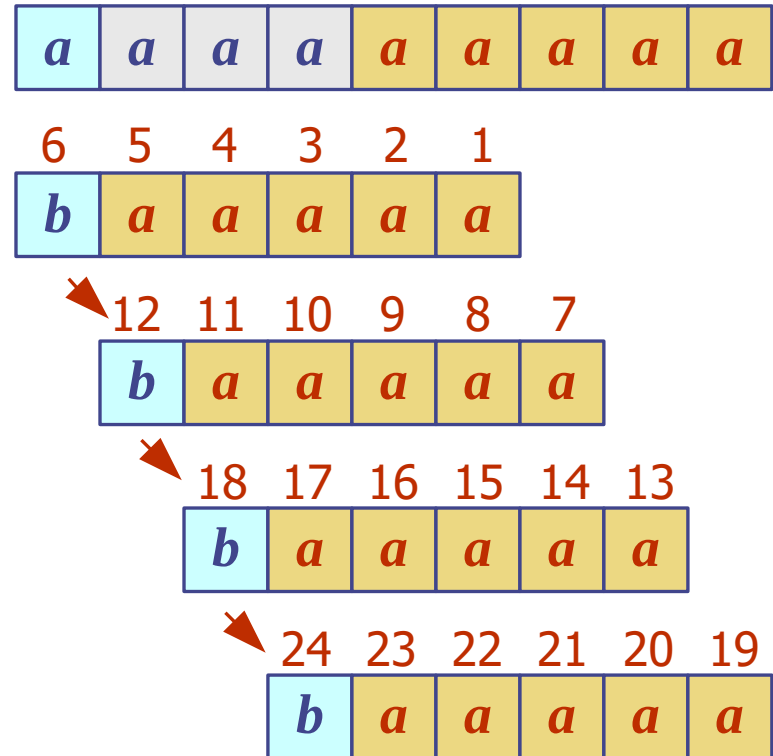
```

If you wish to understand others you must intensify ..
Success!!

Try the Boyer-Moore algorithm with:
text: THIS IS A STRING SEARCH EXAMPLE
pattern: AMPLE

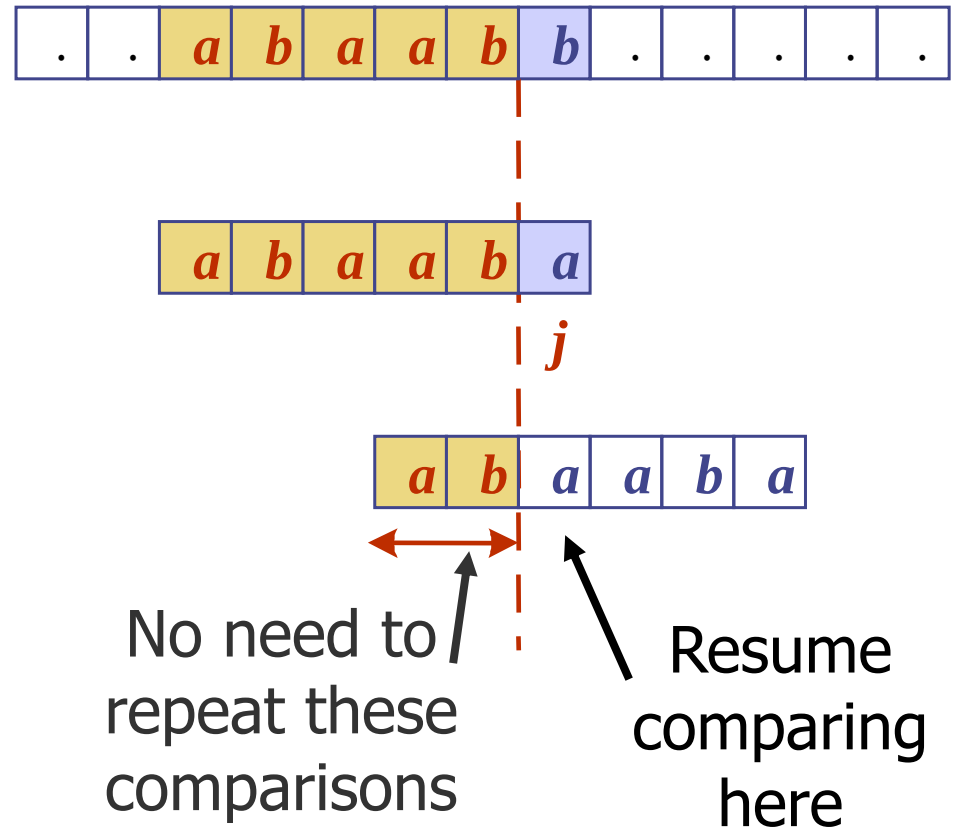
Analysis

- ◆ Boyer-Moore's algorithm runs in time $O(nm + s)$ for worst case
- ◆ Example of worst case:
 - $T = aaa \dots a$
 - $P = baaa$
- ◆ The worst case may occur in images and DNA sequences but is unlikely in English text
- ◆ Boyer-Moore's algorithm is significantly faster than the brute-force algorithm on English text



The KMP Algorithm - Motivation

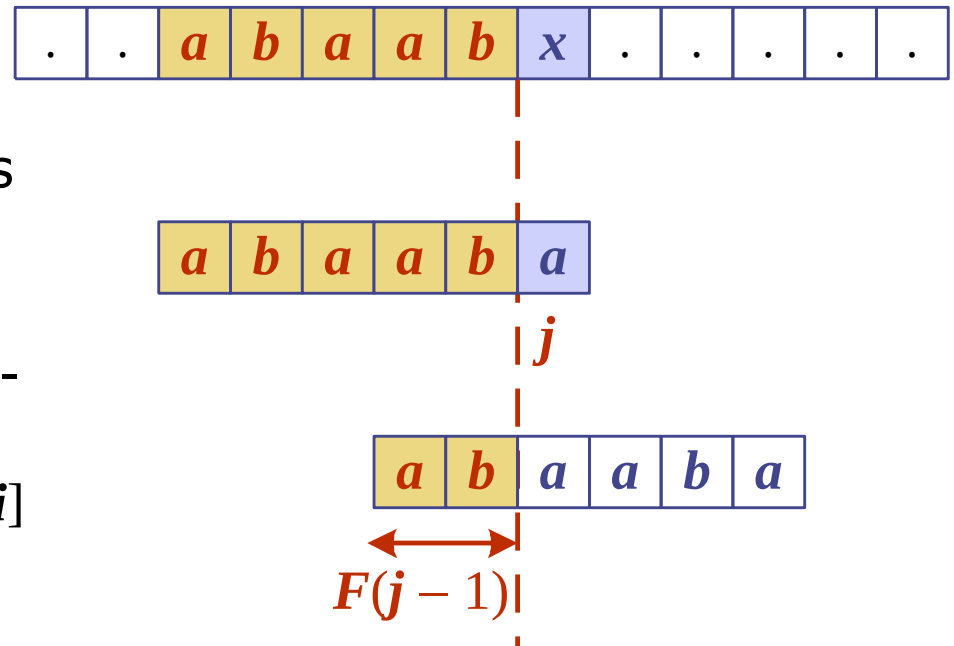
- ◆ Knuth-Morris-Pratt's algorithm compares the pattern to the text in **left-to-right**, but shifts the pattern more intelligently than the brute-force algorithm.
- ◆ When a mismatch occurs, what is the **most** we can shift the pattern so as to avoid redundant comparisons?
- ◆ Answer: the largest prefix of $P[0..j]$ that is a suffix of $P[1..j]$



KMP Failure Function

- ◆ Knuth-Morris-Pratt's algorithm preprocesses the pattern to find matches of prefixes of the pattern with the pattern itself
- ◆ The **failure function** $F(j)$ is defined as the size of the largest prefix of $P[0..j]$ that is also a suffix of $P[1..j]$
- ◆ Knuth-Morris-Pratt's algorithm modifies the brute-force algorithm so that if a mismatch occurs at $P[j] \neq T[i]$ we set $j \leftarrow F(j - 1)$

j	0	1	2	3	4	5
$P[j]$	<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>
$F(j)$	0	0	1	1	2	3



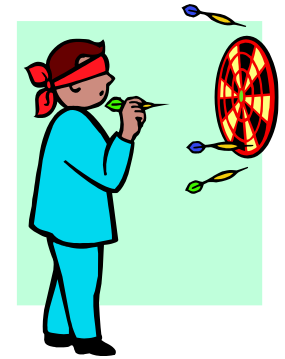
The KMP Algorithm

- ◆ The failure function can be represented by an array and can be computed in $O(m)$ time
- ◆ At each iteration of the while-loop, either
 - i increases by one, or
 - the shift amount $i - j$ increases by at least one (observe that $F(j - 1) < j$)
- ◆ Hence, there are no more than $2n$ iterations of the while-loop
- ◆ Thus, KMP's algorithm runs in optimal time $O(m + n)$

Algorithm *KMPMatch*(T, P)

```
 $F \leftarrow \text{failureFunction}(P)$   
 $i \leftarrow 0$   
 $j \leftarrow 0$   
while  $i < n$   
    if  $T[i] = P[j]$   
        if  $j = m - 1$   
            return  $i - j$  { match }  
        else  
             $i \leftarrow i + 1$   
             $j \leftarrow j + 1$   
    else  
        if  $j > 0$   
             $j \leftarrow F[j - 1]$   
        else  
             $i \leftarrow i + 1$   
return  $-1$  { no match }
```

Computing the Failure Function



- ◆ The failure function can be represented by an array and can be computed in $O(m)$ time
- ◆ The construction is similar to the KMP algorithm itself
- ◆ At each iteration of the while-loop, either
 - i increases by one, or
 - the shift amount $i - j$ increases by at least one (observe that $F(j - 1) < j$)
- ◆ Hence, there are no more than $2m$ iterations of the while-loop

Algorithm *failureFunction*(P)

$F[0] \leftarrow 0$

$i \leftarrow 1$

$j \leftarrow 0$

while $i < m$

if $P[i] = P[j]$

 {we have matched $j + 1$ chars}

$F[i] \leftarrow j + 1$

$i \leftarrow i + 1$

$j \leftarrow j + 1$

else if $j > 0$ **then**

 {use failure function to shift P }

$j \leftarrow F[j - 1]$

else

$F[i] \leftarrow 0$ { no match }

$i \leftarrow i + 1$

Example

a b a c a a b a c c a b a c a b a a b b

1 2 3 4 5 6
a b a c a b

7
a b a c a b

8 9 10 11 12
a b a c a b

13
a b a c a b

14 15 16 17 18 19
a b a c a b

<i>j</i>	0	1	2	3	4	5
<i>P[j]</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>c</i>	<i>a</i>	<i>b</i>
<i>F(j)</i>	0	0	1	0	1	2

Trie

- ◆ Preprocessing the pattern speeds up pattern matching queries
 - After preprocessing the pattern, KMP's algorithm performs pattern matching in time proportional to the text size
- ◆ If the text is large, immutable and searched for often (e.g., works by Shakespeare), we may want to preprocess the text instead of the pattern
- ◆ A trie is a compact data structure for representing a set of strings, such as all the words in a text
 - A trie supports pattern matching queries in time proportional to the pattern size

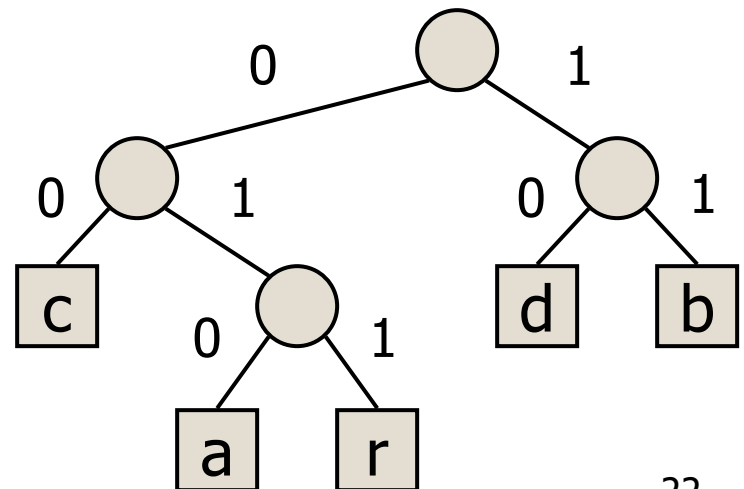
Trie cont.

- ◆ Given a string X, efficiently encode X into a smaller string Y
 - Each uncompressed character in X is 8-bit for EASCII (Extended ASCII) and 16-bit for UNICODE
 - A compressed character in Y has less bit
 - Saves memory and/or bandwidth

Encoding Trie

- ◆ A code is a mapping of each character of an alphabet to a binary code-word
- ◆ A prefix code is a binary code such that no code-word is the prefix of another code-word
- ◆ An encoding trie represents a prefix code
 - Each leaf stores a character
 - The code word of a character is given by the path from the root to the leaf storing the character (0 for a left child and 1 for a right child)

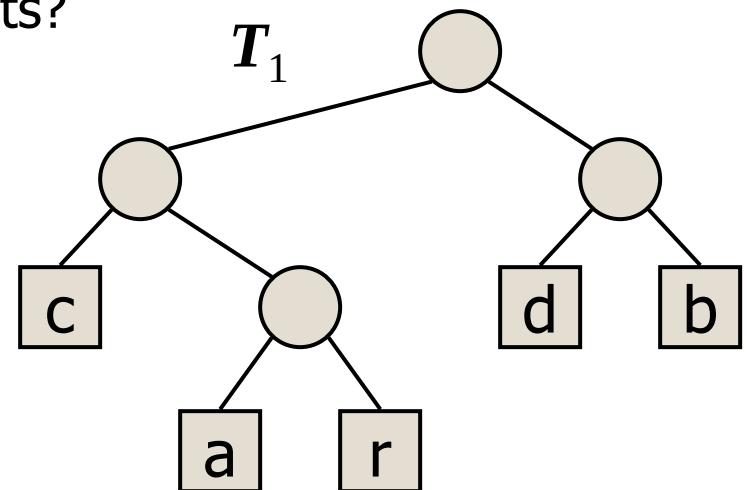
Code word	00	010	011	10	11
Character	c	a	r	d	b



Encoding Trie cont.

- ◆ Given a text string X , we want to find a prefix code for the characters of X that yields a small encoding for X
 - Frequent characters should have short code-words
 - Rare characters should have long code-words
- ◆ $X = \mathbf{abracadabra}$ ($11 \times 8 = \underline{88}$ bits in EASCII)?
- ◆ T_1 encodes X into 29 bits
- ◆ Can X be encoded in less than 29 bits?

Code word	00	010	011	10	11
Character	c	a	r	d	b
Frequency	1	5	2	1	2
Total bit	2	15	6	2	4



Huffman Encoding

- ◆ Compute frequency $f(c)$ for each character c
- ◆ Encode high-frequency characters with short code words (Greedy method)
- ◆ Use an optimal encoding tree to determine the code words

Huffman's Algorithm

- ◆ Given a string X , Huffman's algorithm constructs a prefix code that minimizes the size of the encoding of X
- ◆ It runs in time $O(n + d \log d)$, where n is the size of X and d is the number of distinct characters of X
- ◆ A heap-based priority queue is used as an auxiliary structure

Algorithm *HuffmanEncoding*(X)

Input string X of size n

Output optimal encoding trie for X

$C \leftarrow \text{distinctCharacters}(X)$

$\text{computeFrequencies}(C, X)$

$Q \leftarrow$ new empty heap

for all $c \in C$

$T \leftarrow$ new single-node tree storing c

$Q.\text{insert}(\text{getFrequency}(c), T)$

while $Q.\text{size}() > 1$

$f_1 \leftarrow Q.\text{minKey}()$

$T_1 \leftarrow Q.\text{removeMin}()$

$f_2 \leftarrow Q.\text{minKey}()$

$T_2 \leftarrow Q.\text{removeMin}()$

$T \leftarrow \text{join}(T_1, T_2)$

$Q.\text{insert}(f_1 + f_2, T)$

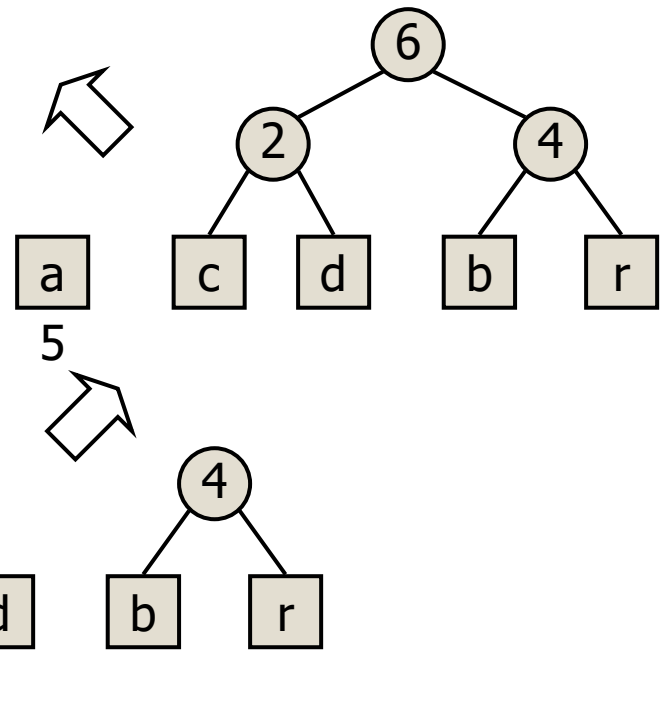
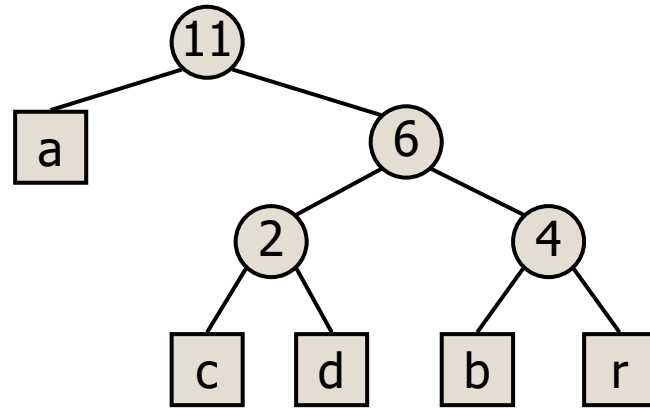
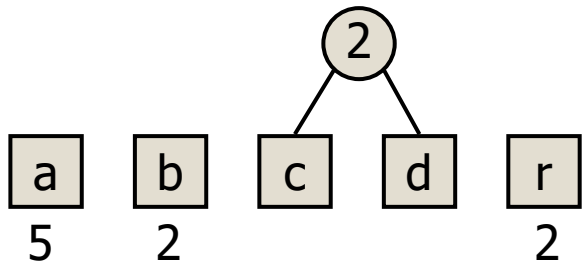
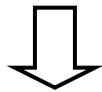
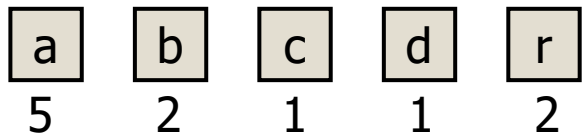
return $Q.\text{removeMin}()$

Huffman's Example

$X = \text{abracadabra}$

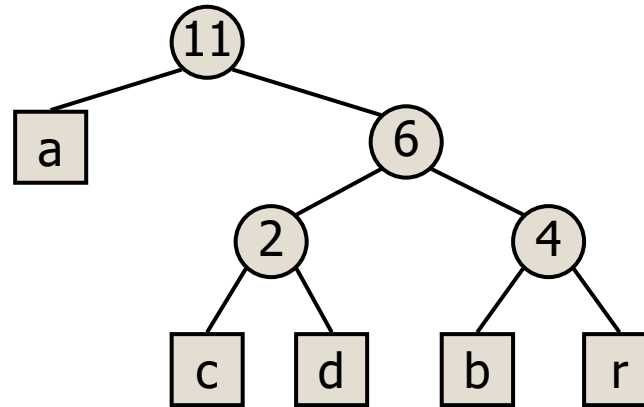
Frequencies

a	b	c	d	r
5	2	1	1	2



Huffman's Example cont.

X = abracadabra



Character	a	b	c	d	r
Frequency	5	2	1	1	2
Code word	0	110	100	101	111
Total bit	5	6	3	3	6

Huffman's algorithm encodes X into 23 bits.

Encoded X = 01101110100010101101110